

普及组模拟 25

一. 单选题

- 2021 年是辛丑年, 下一个己亥年是 ()
A. 2031 年 B. 2043 年 C. 2079 年 D. 2091 年
 - 下列哪一项与其他不相等 ()。
A. 26 (16) B. 46 (8) C. 210 (4) D. 100110 (2)
 - 用 1 米长的竹竿截取 30 厘米和 40 厘米的短竹竿各 100 根。在尽量节省竹竿的情况下, 至少要用到 () 根长竹竿。
A. 76 根 B. 83 根 C. 75 根 D. 84 根
 - 以下关于二分查找算法的描述中, 不正确的是 ()。
A. 二分查找算法的最大查找时间与查找对象的大小成正比
B. 二分查找一般从数组的中间元素开始
C. 二分查找只对有序数组有效
D. 二分查找可以使用递归实现
 - 10 张椅子放成一排, 5 人就坐, 恰有 4 个连续空位的坐法有 () 种
A. 480 B. 960 C. 3600 D. 1800
- 第 6 道题 (2 分)
- 现有 A, B, C, D, E, F, G 七个元素依次进栈操作, 但可随时出栈, 出了栈后 不能再进栈, 下面哪个是不可能的
A. $ABCDEF G$ B. $ADBCGF E$ C. $ABCDEF G F$ D. $GFEDCBA$
 - 二进制补码求和: $11010010 + 00000101 =$
A. 11000011 B. 10110110 C. 11010111 D. 10001001
 - 若让元素 1, 2, 3, 4, 5 依次进栈, 则出栈次序不可能出现 () 的情况。
A. 5, 4, 3, 2, 1 B. 2, 1, 5, 4, 3 C. 4, 3, 1, 2, 5 D. 1 2 5 4 3
 - 中缀表达式 $A - (B + C/D) * E$ 的后缀表达式是 ()
A. $AB - C + D/E *$ B. $ABC + D/ - E *$ C. $ABCD/E * + -$ D. $ABCD/ + E * -$
 - 一个班级共有学生 48 人, 其中 27 人会游泳, 33 人会骑自行车, 40 人会打乒乓球, 那么这个班级至少有 () 名学生这三项运动都会。
A. 5 B. 6 C. 4 D. 8
 - 一个三位数的百位是十位的两倍, 十位比各位大 3. 将这个数的三位加起来的和可能是 ()。
A. 12 B. 13 C. 15 D. 18
 - 一个 14 人的足球队中要选出一位队长和一位副队长。假设球队中的每个人都可以担任这两个职位, 那么一共可以有 () 种不同的队长组合。
A. 48 B. 96 C. 182 D. 256

13. 以下哪一项不属于栈的方法 ()。

- A. *push()* B. *pop()* C. *back()* D. *top()*

14. 两个箱子中分别有 20 和 50 个小球,两人轮流在其中一个箱子中取任意 (1 - 4) 个球,谁先取完其中一个箱子谁赢。

- A. 先取的必赢。 B. 后取的必赢。 C. 都不一定。 D. 哼

15. 我们身边的电脑和电子产品计算时主要使用 ()。

- A. 十六进制 B. 二进制 C. 十进制 D. 八进制

二. 阅读程序

阅读下面的程序完成相应的题目

1. 程序阅读题 1

```
1 #include <iostream>
2 using namespace std;
3 int main()
4 {
5     int n, num[3];
6     cin >> n;
7     string s;
8     cin >> s;
9     num[1] = n / 2;
10    num[2] = n / 2;
11    int sum = 0;
12    for (int i = 0; i < s.length(); i++)
13    {
14        if (s[i] == '1')
15            sum++;
16    }
17    if (sum <= n)
18    {
19        for (int i = 1; i <= n - sum; i++)
20            s = s + '1';
21    }
22    for (int i = 0, j = 1; i < s.length(); i++)
23    {
24        if (s[i] == '1')
25        {
26            num[j]--;
27            j = j % 2 + 1;
28        }
29        if (num[1] == 0 || num[2] == 0)
30            break;
```

```

31         j = j % 2 + 1;
32     }
33     if (num[1] == 0)
34         cout << "BWin";
35     else
36         cout << "AWin";
37     return 0;
38 }

```

注：字符串由 01 构成

16. 当输入字符串全为 1 时 一定是 "A Win"; ()

17. 当输入字符串全为 0 时 一定是 "A Win"; ()

第 3 道题 (2 分)

18. 将程序第 17 行改为 $sum \leq num[1] + num[2]$, 19 行条件改为 $i \leq num[1] + num[2] - sum$; 不会对程序发生影响 ()

19. 输入时 n 必须大于等于 0 否则程序会出错: ()

20. 以下那种情况可以确保答案一定时 "A Win" ()

A. 字符串 01 交替, $sum < n$

B. 字符串 10 交替, $sum < n$

C. 字符串 01 交替, $sum > n$

D. 字符串 10 交替, $sum > n$

2. 程序阅读题 2

```

1、#include <iostream>
2、using namespace std;
3、const int N = 100005;
4、int n, m, a[N];
5、int f(int l, int r)
6、{
7、    if (r < l)
8、        return 0;
9、    int mid = (l + r) / 2;
10、    int max = 0;
11、    for (int i = l; i <= r; i++)
12、    {
13、        if (a[i] > max)
14、        {
15、            max = a[i];
16、        }
17、    }
18、    if (max == a[mid])
19、        return 1;
20、    else
21、        return f(l, mid - 1) + f(mid + 1, r);

```

```

22、}
23、int main()
24、{
25、    cin >> n >> m;
26、    for (int i = 1; i <= n; i++)
27、    {
28、        cin >> a[i];
29、    }
30、    int sum = f(1, n);
31、    cout << sum << endl;
32、    if (sum >= m)
33、        cout << "yes";
34、    else
35、        cout << "no";
36、    return 0;
37、}

```

21. 如果输入时序列为升序,那么最后 sum 一定是 $\text{int}(\log(n))$ 。

()

22. 假设 p 是任意正整数, sum 一定是 2^p ()

23. 当序列数全部相同时, mid 和 max 位置不等, sum 将取得最大值()

24. 程序的最坏复杂度是()

A. $\log(n)$ B. $n * \log(n)$ C. $n * n$ D. $\log(n) * \log(n)$

25. 如果序列以 2 1 进行交替任意 2^p 次 2 1 2 1.....2 1,那么该程序的 sum 结果是 (), (p 是任意正整数)

A. 1 B. $\log(n)$ C. $n/2$ D. 不能确定

3. 程序阅读题 3

```

1、#include <iostream>
2、#include <string>
3、using namespace std;
4、const int N = 105;
5、int n, m, x, y, cnt, cx[] = {1, -1, 0, 0}, cy[] = {0, 0, 1, -1};
6、char a[N][N];
7、string s;
8、bool vis[N][N];
9、int main()
10、{
11、    cin >> n >> m;
12、    for (int i = 1; i <= n; i++)
13、    {

```

```

14、    for (int j = 1; j <= m; j++)
15、    {
16、        cin >> a[i][j];
17、        if (a[i][j] == '*')
18、        {
19、            vis[i][j] = true;
20、            cnt++;
21、        }
22、        if (a[i][j] == 'T')
23、        {
24、            x = i;
25、            y = j;
26、        }
27、    }
28、}
29、cin >> s;
30、for (int i = 0; i < s.length(); i++)
31、{
32、    for (int j = 0; j < 4; j++)
33、    {
34、        if (s[i] - '0' == j + 1)
35、        {
36、            int
37、                dx = x + cx[j];
38、            int
39、                dy = y + cy[j];
40、            if (vis[dx][dy] == true)
41、            {
42、                vis[dx][dy] = false;
43、                cnt--;
44、            }
45、            x = dx;
46、            y = dy;
47、        }
48、    }
49、}
50、cout << x << ' ' << y << ' ' << cnt;
51、return 0;
52、}

```

注： s 由 '1','2','3','4' 构成。

26. 输入的字符串 s 必须是由 '1','2','3','4'。 组成否者程序将无法运行。()

27. 如果第 12 行循环结束时 $cnt = n * m - 1$ 。 只要 s 的长度不为 0 那么最后输出 的结果 $cnt < n * m$

-1。()

28. 将 40 行的 `vis[dx][dy]==true` , 改写为 `a[dx][dy]=='*` 对程序不会产生影响。()

29. 如果 n 和 m 足够大, $x=n/2, y=n/2, s$ 按照 12341234....1234 去走, 在第 74 个字符时处在什么位置。
()

A. $x+1, y$ B. x, y C. $x, y+1$ D. $x+1, y+1$

30. 假如 $n=5, m=5, cnt=24$, 如果 s 由 5 个 1, 5 个 2, 5 个 3, 5 个 4 组成, 那么在不超出边界的情况下 cnt 的范围是()

A. 5~21 B. 4~20 C. 6~22 D. 4~22

三. 完善程序

1. 完善程序 1

高铁是我国重要的通行方式, 这边的话我们给出 N 个闭环来模拟道路情况, 当然每个闭环所构成的点数量肯定是不一样的, 当然也存在一些交通要道, 这些点可能属于多个环中, 但是铁轨只能通向一个方向这个时候就需要改变铁轨的方向。现在的问题是从 1 号点出发到达 k 点需要改变铁轨的最少次数, 铁轨只能单项通行。

```
1、    #include <iostream>
2、    #include <vector>
3、    Using namespace std;
4、    Const int N = 1005;
5、    int n, k, ans;
6、    bool vis[N];
7、    struct node
8、    {
9、        int next;
10、       vector<int> y;
11、    } g[N];
12、    void dfs(int x, int cnt)
13、    {
14、        if (x == k)
15、        {
16、            ans = min(ans, cnt);
17、            return;
18、        }
19、        for (int i = 0; ①; i++)
20、        {
21、            int y =②;
22、            if (vis[y] == true)
23、                continue;
24、            vis[y] = true;
25、            if (③)
26、                dfs(y, cnt + 1);
```

```

27、         else
28、             dfs(y, cnt);
29、         vis[y] = false;
30、     }
31、 }
32、 int main()
33、 {
34、     cin >> n >> k;
35、     for (int i = 1; i <= n; i++)
36、     {
37、         int m;
38、         cin >> m;
39、         for (int j = 1; j <= m; j++)
40、         {
41、             int x, y;
42、             cin >> x >> y;
43、             g[x].next = ④;
44、             ⑤;
45、         }
46、     }
47、     dfs(1, 0);
48、     cout << ans;
49、     return 0;
50、 }

```

31. 1 号位置填 ()

A. $i < g[x].size()$ B. $i < g[x].next$ C. $i < g[x].y.size()$ D. $i <= g[x].next$

32. 2 号位置填 ()

A. $g[x].y[next]$ B. $g[x].next$ C. $g[x].y[i]$ D. i

33. 3 号位置填 ()

A. $g[x].next \neq i$ B. $g[x].next \neq 0$ C. $g[x].next \neq y$ D. $x \neq y$

34. 4 号位置填 ()

A. i B. j C. 0 D. m

35. 5 号位置填 ()

A. $g[y].next = x$ B. $g[x].y.push_back(y)$
C. $g[x].y[j] = 1$ D. $g[x].y[y] = 1$

2. 程序填空题 2

蜘蛛纸牌是一款电脑上的纸牌游戏,现在有一堆纸牌,一开始我们分为 8 堆,每堆 10 张,只有最顶上的牌是看的见,想要看到下面的牌就必须把上面的拿掉。现在我们每次可以移动这些牌,但是每次只能移动到比自身大 1 的牌下方。当其中一堆牌为空时,这个时候可以移动任意牌到这里,如果牌连号那么可以一起移动。

获胜条件：数字 1 到数字 10 为一组只要能组成 10 个数连号就能移出去，80 张牌全部移出就获胜，移动次数 400 以内。那么游戏一开始我们只能对每堆的牌顶进行操作，而下面的牌全是背面朝上。只有在上面牌移走后会才会转过来，就算后面有牌发上来也不会变回去。那么这里我们采用栈来模拟每一对牌的情况。

```
1、#include <iostream>
2、#include <vector>
3、using namespace std;
4、int st[11][81];
5、int len[11], top[11]; // len 表示每堆看不到的牌的数量, top 表示当前堆一共的牌数
6、int main()
7、{
8、    int n = 8, m = 10;
9、    for (int i = 1; i <= n; i++)
10、    {
11、        for (int j = 0; j < m; j++)
12、        {
13、            cin >> st[i][j];
14、        }
15、        top[i] = m;
16、        len[i] = m - 1;
17、    }
18、    int num = 80, cnt = 0;
19、    while (num != 0)
20、    {
21、        for (int i = 1; i <= n; i++)
22、        {
23、            int sum = 1;
24、            for (int j = top[i] - 1; j >= 0; j--)
25、            {
26、                if (st[i][j] == st[i][j - 1] - 1 && j >= len[i])
27、                    sum++;
28、            }
29、            for (int j = 1; j <= n; j++)
30、            {
31、                if (i == j)
32、                    continue;
33、                if (top[j] == 0 || ① )
34、                {
35、                    for (int k = sum; k >= 1; k--)
36、                    {
37、                        st[j][top[j]] = st[i][top[i] - sum];
38、                        ②
39、                    }
40、                    sum = 1;
```



```

41、                for (int k = top[j] - 1; ③; k--)
42、                {
43、                    if (st[j][k] == st[j][k - 1] - 1)
44、                        sum++;
45、                }
46、                if (sum == 10)
47、                {
48、                    ④
49、                }
50、                if (top[i] == len[i])
51、                {
52、                    ⑤
53、                }
54、                break;
55、            }
56、        }
57、    }
58、    cnt++;
59、    if (cnt == 400)
60、    {
61、        cout << "Lose";
62、        return 0;
63、    }
64、}
65、cout << "Win";
66、return 0;
67、}

```

36. 1 号空填 ()

- A. $st[j][top[j] - 1] == st[i][top[i] - sum - 1] - 1$
- B. $st[j][top[j] - 1] - 1 == st[i][top[i] - sum - 1]$
- C. $st[j][top[j] - 1] - 1 == st[i][top[i] - sum]$
- D. $st[j][top[j] - 1] == st[i][top[i] - sum] - 1$

37. 2 号空填 ()

- A. $top[j]++$; $top[i]--$;
- B. $top[i]++$; $top[j]--$;
- C. $len[i]++$; $len[j]--$;
- D. $len[j]++$; $len[i]--$;

38. 3 号空填 ()

- A. $k \geq 0$
- B. $k \geq 1$
- C. $k > len[j]$
- D. $k \geq len[j]$

39. 4 号空填 ()

- A. $top[j] -= 10$; $num -= 10$;
- B. $top[i] -= 10$; $num -= 10$;
- C. $top[i] -= 10$; $top[j] += 10$;
- D. $top[j] = 0$; $num -= 10$;

40.5 号空填()

A. $len[i] = 0;$

B. $len[i] --;$

C. $len[i] = \max(len[i] - 1, 0);$

D. $len[i] = \max(len[i] - 1, 1);$